

3.58 msortf レコードの並べ換え

f=パラメータで指定した項目を基準にして、レコードを並べ換える。

ソーティングアルゴリズムは quick sort を利用しており、安定ソート (キーの値が同じ行については元の順序を保存する) にはならないことに注意する。

書式

```
msortf f= [pways=] [maxlines=] [blocks=] [threadCnt=] [-noflg] [i=] [o=] [-assert_diffSize] [-nfn] [-nfno] [-x] [-q] [tmpPath=] [--help] [--help1] [--version]
```

パラメータ

f= レコードを並べ換える基準となる項目名リストを指定する。

並び順は、数値/文字列、昇順/降順の組み合わせで 4 通り指定できる。

指定方法は % に続けて **n** と **r** を以下の通り組み合わせる。

文字列昇順:項目名 (% 指定なし)、文字列逆順:**f**=項目名 **%r**、数値昇順:**f**=項目名 **%n**、数値降順:**f**=項目名 **%nr**。

備考

1. **k**=で、文字列項目に対して **%n** を指定した場合の動作は不定である。
2. **k**=を省略した場合は **i**=で指定したファイルを順番に併合する (**mcat** と同様)。
3. キー項目に NULL 値が含まれる場合、NULL 値はどのような値よりも小さい値として扱われる。
4. **i**=で指定したファイルは全て存在し、また項目名は全て同じであることを前提としており、**mcat** のような柔軟な指定はできない。

利用例

例 1: 基本例

item、date 順に並べ替える。

```
$ more dat1.csv
item,date,quantity,price
B,20081201,4,40
A,20081201,10,200
A,20081201,10,100
B,20081203,5,50
B,20081201,2,500
A,20081201,3,300
$ msortf f=item,date i=dat1.csv o=rs11.csv
#END# kgsortf f=item,date i=dat1.csv o=rs11.csv
$ more rs11.csv
item%0,date%1,quantity,price
A,20081201,10,200
A,20081201,10,100
A,20081201,3,300
B,20081201,4,40
B,20081201,2,500
B,20081203,5,50
```

例 2: 数量 (quantity) 降順, 金額 (price) 昇順に並べ替える例

```
$ msortf f=quantity%nr,price%n i=dat1.csv o=rsl2.csv
#END# kgsortf f=quantity%nr,price%n i=dat1.csv o=rsl2.csv
$ more rsl2.csv
item,date,quantity%0nr,price%1n
A,20081201,10,100
A,20081201,10,200
B,20081203,5,50
B,20081201,4,40
A,20081201,3,300
B,20081201,2,500
```

上級者用パラメータ

pways= 同時併合ファイル数 ([2-100]:デフォルト 32) 【任意】
分割ソートされた複数のファイルを同時に何個併合するかを指定する。

blocks= バッファブロック数 ([1-1000]:デフォルト 10) 【任意】
メモリ内でソートする際のメモリサイズ上限をブロックサイズで指定する。
1 ブロックは入力バッファサイズ×4で、デフォルトは 4MB。

maxlines= メモリソートレコード件数上限 ([100-1000 万]:デフォルト 50 万) 【任意】
メモリ内でソートする際の件数の上限を指定する。
データの一行あたりの平均サイズに応じて、**blocks=**制限と **maxlines=**制限のいずれかが使われる。

threadCnt= メモリ内でソートを実行する thread 数 ([1-50]:デフォルト 8) 【任意】
分割ソートする際に、マルチスレッドの機能を用いて同時にソートする数を指定する。

CSV 特殊文字と並び順についての注意

`msortf` では CSV の特殊文字を解釈して並べ替えを行う為に、CSV の特殊文字であるカンマとダブルクォーツを含むデータについては UNIX の `sort` コマンドの実行結果と異なることがある。例えば、以下に示すような第一項目に `a(0x61)`、NULL 値、スペース文字 (`0x20`)、`+(0x2b)`、`-(0x2d)`、カンマ (`0x2c`)、ダブルクォーツ (`0x22`) を持つデータについて見てみよう。カンマとダブルクォーツは CSV の特殊文字のため特別な表記となっている。また表示上の分かりやすさのために第二項目 `f2` として全行に”x”を付加している。

```
f1,f2
a,x
,x
,x
+,x
-,x
",",x
""",x
```

このデータを「`msortf f=f1`」で並べ替えた結果は以下の通りで、CSV フォーマットにおける特殊記号を考慮した上での本来の並び順 (NULL, スペース, ダブルクォーツ, +, カンマ, -, a) になっていることがわかる。

```
f1,f2
,x
,x
""",x
+,x
",",x
-,x
a,x
```

ベンチマークテスト

msortf の速度についてのベンチマークテストの結果を示す。入力データは以下に示すような 6 項目からなるデータである。全ての項目は一様乱数により生成されている。

```
key,fld1,fld2,fld3,fld4,fldn
95547922,162,159,192,118,74
81438069,138,157,155,122,58
26885062,129,199,133,198,75
32651684,180,107,123,170,-14
10245631,164,103,159,154,-63
15145156,182,191,175,107,-60
29254245,188,185,129,124,5
85423170,116,164,175,113,57
55155879,105,163,195,167,25
66997216,195,139,195,113,39
.
.
```

キー項目の値の種類数および状態に関する比較

レコード数を 100 万件とし、キー (key 項目) の値の種類を 2、10、100、1000、10000 とした場合の結果を表??に示す。また、表中の (1),(2) をグラフ化したものが図 3.3 に、また (4),(5) をグラフ化したものが図 3.4 に示されている。

「乱数」項目は乱数の上限を最大とした場合の乱数値をキーとしている。また「乱数昇順 (降順)」は、「乱数」データを事前に昇順 (降順) に並べ替えたデータである。msortf 以外に、比較の為に MUSASHI の xtsort コマンド、および UNIX の sort コマンドの結果も示す。「sort -k1」は、第 1 番目の項目で並べ替えた結果である。また上三行は msortf、xtsort、sort の各コマンドで第 1 番目の項目を文字列として並べ替えた時の結果であり、下三行は数値として並べ替えたときの結果である。実行環境は、MacBookPro, Mac OS X 10.9.1, 2.6GHz Intel Core i7, 16GB メモリ、である。

表 3.31 キー項目の値の種類数および状態に関する比較

No.	コマンド	2 種	10 種	100 種	1000 種	10000 種	乱数	乱数昇順	乱数降順
(1)	msortf f=key	0.29	0.33	0.37	0.40	0.43	0.50	0.29	0.28
(2)	xtsort -k key	1.25	1.24	1.22	1.20	1.19	1.12	0.85	1.00
(3)	sort -k1	16.96	16.63	16.05	15.56	15.08	13.68	6.85	7.13
(4)	msortf f=key%n	0.46	0.56	0.65	0.72	0.79	1.02	0.59	0.59
(5)	xtsort -k key%n	2.52	2.72	2.96	3.16	3.21	3.22	2.31	2.32
(6)	sort -k1 -n	16.65	14.52	11.54	8.56	5.71	0.95	0.33	0.36

msortf は xtsort より 2 倍～5 倍高速である。sort については、条件にもよるが数十倍高速である。MUSASHI の sort とで利用している quick sort のアルゴリズムは全く同じであるが、MCMD においては分割ソートにおいてマルチスレッドを使い並列処理している。その影響の差が出ているということであろう。次に、キーの種類数を 100 および最大に固定し、データ件数を 100 万件から 1000 万件まで変化させた時の文字ソートの速度について実験した。ここでは msortf、xtsort の 2 つのコマンドの比較を図 3.5,3.6 に示す。

関連コマンド

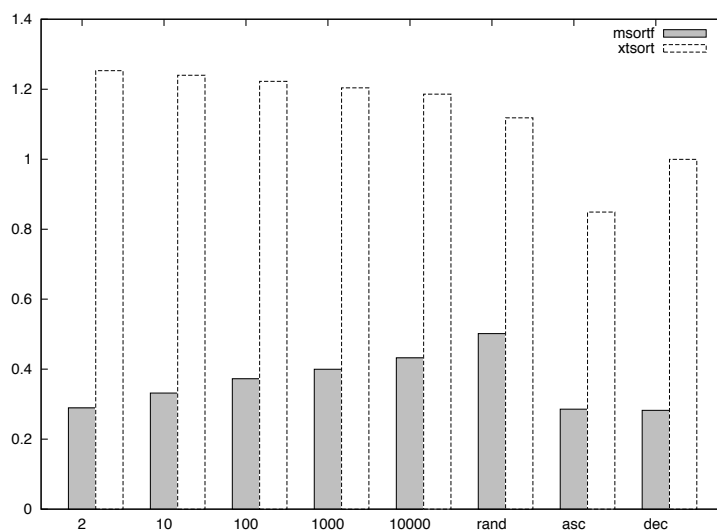


図 3.3 キーの種類数別に見る `msortf`, `xtsort` の文字列ソートの比較 (横軸がキーの種類数, 縦軸が秒数)

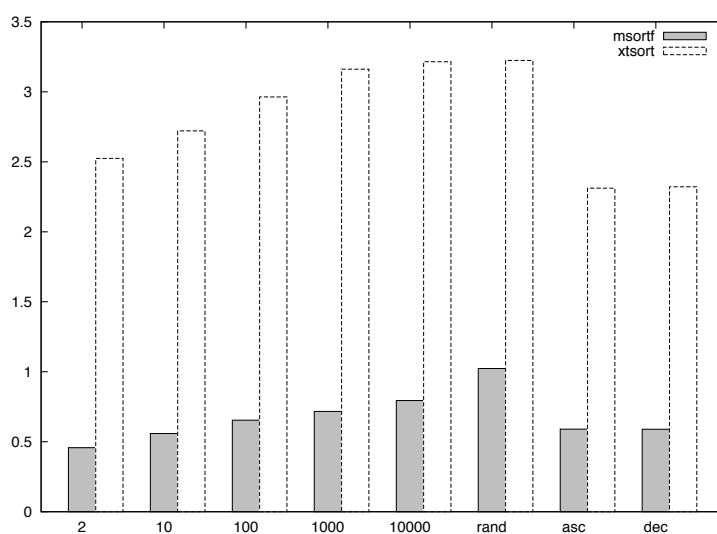


図 3.4 キーの種類数別に見る `msortf`, `xtsort` の数値ソートの比較 (横軸がキーの種類数, 縦軸が秒数)

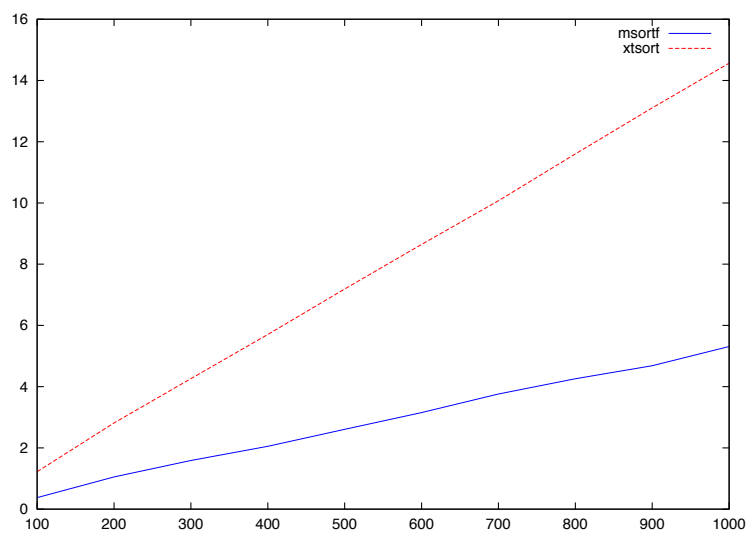


図 3.5 キーの種類数を 100 とした場合の結果 (横軸がデータ件数, 縦軸が秒数)

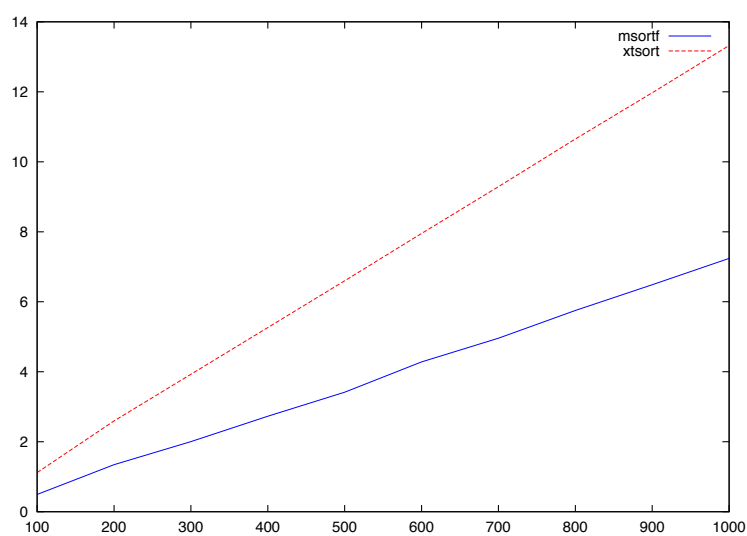


図 3.6 キーの種類数を乱数 (最大) とした場合の結果 (横軸がデータ件数, 縦軸が秒数)